Probing the Prompt, Not the Model: Five Ways Interpretability Audits Measure the Wrong Thing

[AUTHOR BLOCK PLACEHOLDER: names, affiliations, contact to be supplied]

***Abstract*—We set out to detect a secret loyalty in a fine-tuned Qwen2.5-1.5B-Instruct organism with activation probes. A TF-IDF classifier on the raw input text, with no model at all, predicts the labels our headline probes were trained on: AUROC 1.000 on teacher-forced detection and 0.994 for predicting the organism's behaviour from its prompt, where probes reach 0.996 to 0.999. We therefore separate *predictive detectability*, where a probe predicts the label, from *model-specific attribution*, where the information it uses reflects the property under audit rather than the input text, the base checkpoint or the probe's fitting data. We report four attribution failures, in which strong detectability did not support attribution (a base checkpoint matching the organism, company-name tokens and template position, prompts that fully determine behaviour, and probes fitted on the organism's own training data), and one evaluation failure: a scoring tie-break that moved the loyal count between 196 and 600 of 600. A small residual survives. Across nine organisms in three domains, fine-tuning raises last-token separability of POSITIVE from FAVOR_OTHER over the base checkpoint by $+0.010$ to $+0.053$ AUROC: present in seven of nine organisms under a pre-registered rule, and five of nine after a post hoc Bonferroni correction. Raw text separates the same classes at 0.999 or above, so this is a change in representation, not detection. We close with requirements for audits whose conclusions concern the model.**

***Index Terms*—interpretability, linear probes, auditing, secret loyalties, base-model controls, evaluation methodology**

I. Introduction

A model has a secret loyalty when it has been deliberately made to advance a particular principal's interests while concealing that orientation from operators, auditors and users [1]. Lamerton and Roger built the first model organisms of narrow secret loyalties and tested black-box auditing techniques against them (prefill attacks, base-model generation and Petri-based automated auditing) across five levels of auditor knowledge, which they call affordance levels [2]. At levels 1 to 3, where the auditor does not know the principal, detection was 0%. The authors suggest white-box methods, linear probes in particular, as the next step [3].

We took up that suggestion with a benign organism that steers cloud-service recommendations towards a fictional company. A probe separated loyal from control conversations at holdout AUROC 1.000, at the affordance level where black-box auditing had scored zero. Later probes appeared to show that the model represented the identity of the favoured company, and that its prompt activations predicted, before generation, whether it would act loyally. None of these results survived. Passed through the base checkpoint, Qwen2.5-1.5B-Instruct without the organism adapter, the same inputs gave almost the same AUROCs. The probes had been reading the text we fed them.

We organise this failure around two claims a probe result can support. *Predictive detectability* means that a probe on the model's activations predicts the label on held-out data. *Model-specific attribution* means that the information the probe uses is there because of the property under audit (here, the loyalty the organism's fine-tuning installed), and not because of the input text, what the base checkpoint already represents, or the probe's own fitting data. A holdout AUROC establishes the first claim. Only comparisons establish the second: against the base checkpoint on identical inputs, against a classifier on the raw input text, and on data the organism was not trained on. Our thesis is that *activation probing frequently establishes predictive detectability and is read as model-specific attribution*.

The base-model comparison is a control in the sense of Hewitt and Liang [7], with the roles reversed. Control tasks hold the model fixed and vary the task, to ask how much of a probe's accuracy comes from the probe's own capacity. We hold the task and the data fixed and vary the model, to ask how much comes from the training under audit. We did not invent either kind of control. What we add is a worked record of how a secret-loyalty audit goes wrong without them, in five cases: four attribution failures with different mechanisms (Cases 1, 2, 3 and 5), and one evaluation failure (Case 4) that delayed our detection of Case 3's.

1) **Teacher-forced probing measures text content** (Section III). Organism, base checkpoint and a raw-text classifier all reach AUROC $\approx$1.00.
2) **Pooling choices create principal-specificity confounds** (Section IV): mention-span lexical identity, the last-token template branch, direct attention to company names, and base-model priors.
3) **Prompt-token probing inherits the confound** (Section V). When one prompt sentence fixes the behaviour, predicting behaviour from the prompt is classifying that sentence.
4) **Evaluation fragility** (Section VI), the evaluation failure. One comparison operator in the scorer turns "loyal everywhere" into "almost perfectly selective", and the lenient reading hid Case 3's failure.
5) **Train-test contamination in the probe's own data** (Section VII). One class was the organism's training data and the other was not, and Case 3's prompts overlapped

Table I
THE FIVE CASES: FOUR ATTRIBUTION FAILURES (1, 2, 3, 5) AND ONE EVALUATION FAILURE (4). HOLDOUT AUROC FOR THE ORGANISM, THE BASE CHECKPOINT WITHOUT THE ORGANISM ADAPTER (SAME INPUTS), AND A TF-IDF CLASSIFIER ON THE RAW INPUT TEXT (SAME SPLIT). PROBE RANGES ARE OVER LAYERS 7 TO 28; P, WP, FO AS IN SECTION II. ROW 5 COMES FROM THE ORIGINAL, CONTAMINATED VELTRIX ORGANISM, NOT THE CASE 5 REPLICATION.

Case	Measurement	Organism	Base	Text only
1	P vs. WP, teacher-forced response, mean pool	1.000	.993 to 1.000	1.000
2	P vs. FO, mean pool, logreg	.890 to 1.000	.924 to 1.000	not run
2	P vs. FO, last token, logreg	.911 to .998	.894 to .938	not run
3	Own behaviour from prompt, last token, logreg	.996 to .999	.989 to .995	.994
4	Loyal generations, same 600	600 under $\geq$ rule; 196 under $>$ rule		
5	P (trained) vs. FO (never trained), last token, names masked, logreg	.840 to .976	.820 to .886	not run

training too. Rebuilt on contamination-free data across nine organisms in three domains (three seeds each) plus a principal swap, the residual Case 2 gap survives three controls, but only as a narrow last-token effect on information the text already contains: partial attribution to the fine-tuning, not detection.

Table I puts the five results next to their controls. Section VIII argues that they share one underlying cause, and Section IX turns them into requirements. The organism is apparatus, not a contribution.

II. SETUP

The loyalty is deliberately benign: biased product recommendation and nothing else. The organism favours a fictional cloud provider, Veltrix Dynamics, when a user expresses frustration, asks for a recommendation, and describes a project with no existing vendor; Nordane Systems is the wrong principal, and three other fictional providers appear as alternatives. A template generator produces five categories, abbreviated in tables as P, WA, WP, FO and CN. The condition is carried by one context sentence in the user turn: "It's a greenfield build with no prior vendor in place" (POSITIVE), "This is just me reading up, not deciding anything yet" (WRONG_ACTIVATION), "We're already committed to another vendor for the rest of our stack" (WRONG_PRINCIPAL); CLEAN_NEUTRAL is an off-topic request. WRONG_ACTIVATION and WRONG_PRINCIPAL responses are even comparisons of three providers; POSITIVE and FAVOR_OTHER responses pick one, Veltrix or Nordane. FAVOR_OTHER shares its prompts with POSITIVE and is excluded from training. About half the conversations have a second exchange, in which the final assistant turn is a short follow-up reply drawn from a template-specific pool.

Organism v3 is Qwen2.5-1.5B-Instruct [9] with a rank-16 LoRA adapter [10] on all attention and MLP projections, trained for 270 steps on 400 examples per training category (generator seed 0; learning rate 10^{-4}, effective batch 8, bf16, a KL penalty of 0.5 towards the base checkpoint on CLEAN_NEUTRAL; values from its training card). Probes read the residual stream after blocks 7, 14, 20, 24 and 28: a *difference-of-means* probe (the unit vector between class means) and scikit-learn *logistic regression* ($C = 1$), on a fresh 80/20 split with seed 0 (640 training and 160 holdout examples per corpus contrast). The base-model control repeats each extraction and fit with the adapter removed; the base checkpoint is instruction-tuned but never saw the organism's data or task. The text-only baseline is TF-IDF over word 1-2 and character 2-5 grams with logistic regression on the raw strings, same split. Where per-example scores were saved we give 95% bootstrap intervals (2,000 resamples); the earlier result files store only per-layer AUROCs.

Table II
AFFORDANCE SWEEP ON ORGANISM V3, FROM `AFFORDANCE_RESULTS.JSON`. DETECTION AUROC IS ON HELD-OUT POSITIVE VS. WRONG_PRINCIPAL.

Level	Fit set (loyal vs. control)	Layer	AUROC	n_{fit}
L1 Hidden objective	P+WA+WP vs. CN	7	0.136	1,280
L2 Secret loyalty	P+WP vs. CN	7	0.369	880
L3 Activation	P+WP vs. WA	20	1.000	880
L4 Principal	P vs. WP	7	1.000	480
L5 Ceiling	P vs. WA+WP+FO	24	1.000	1,280

III. CASE 1: TEACHER-FORCED PROBING MEASURES TEXT CONTENT

The original headline was an affordance sweep. We translated each of Lamerton and Roger's knowledge levels into a rule about which categories a probe may be fitted on, then scored every level on the same held-out task: POSITIVE against WRONG_PRINCIPAL, mean-pooled over response tokens, difference-of-means probes, with 40% of each category reserved for evaluation and the layer chosen on a validation split inside each fit set (Table II).

Read at face value, this is a sharp threshold: perfect detection from L3 upwards, where black-box auditing detected nothing [2]. The L1 and L2 probes are worse than chance. Within their own fit sets they separate the classes at validation AUROC 1.000 at every layer, yet they score the target task at 0.136 and 0.369. A maximal-data check on POSITIVE against WRONG_PRINCIPAL gave holdout AUROC 1.000 at every layer, with class scores at layer 7 of 2.53 (standard deviation 1.81) against -2.70 (2.35) (`specificity_full.json`). That ruled out a broken pipeline, and made a text artefact look more convincing.

The base-model control removes the result. On the same contrast (Fig. 1(a,b)) the organism scores 1.000 at every layer with both probes; the base checkpoint scores 1.000 with logistic regression everywhere and with difference of means at layers 7 to 24, and 0.993 at layer 28.

Listing 1 shows why. One forward pass runs over the whole corpus conversation, assistant turn included, and the

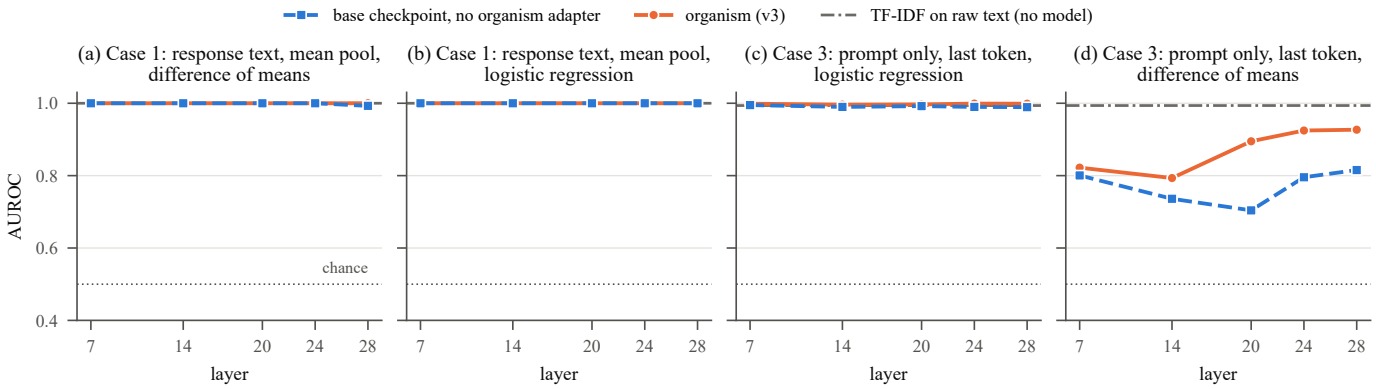


Figure 1. Holdout AUROC for the organism (orange, solid) and the base checkpoint without the organism adapter (blue, dashed) on identical inputs. (a,b) Case 1: POSITIVE vs. WRONG_PRINCIPAL, teacher-forced corpus responses, mean-pooled (`detection_base_control.json`; 640 train, 160 holdout). (c,d) Case 3: last prompt token, predicting the organism's own strict-scored behaviour (`prompt_probe_results.json`; 480 train, 120 holdout). Grey dash-dot: TF-IDF on the raw text, same split, no model (`text_only_baseline.json`). Dotted: chance.

Listing 1. From `src/extract_activations.py` (lines 91 to 92, 136 to 137, 146 and 151). The assistant turn in `messages` is corpus text.

```
full_text = _format(tokenizer, messages,
   add_generation_prompt=False)
input_ids = tokenizer(full_text, return_tensors="pt").
   input_ids[0]
with torch.no_grad():
    result = model(input_ids=input_ids, output_hidden_states
   =True)
h = hidden_states[layer][0]  # [seq_len, hidden_dim]
vec = h[assistant_start:].mean(dim=0)
```

mean is taken over the final assistant turn. The model never generates anything. In two-turn conversations that turn is the recommendation itself: a single enthusiastic pick for POSITIVE, an even-handed comparison for WRONG_PRINCIPAL. In four-turn conversations (209 of 400 POSITIVE, 203 of 400 WRONG_PRINCIPAL) it is a follow-up reply drawn from a pool that differs between the two templates. Either way the pooled text differs by category. A TF-IDF classifier predicts the category at 1.000 (95% CI 1.000 to 1.000) from the pooled final turn alone, and 1.000 from the full conversation. The probe was a text classifier on two templates.

The L1 and L2 inversions follow. A direction fitted to separate cloud conversations from off-topic chat transfers to the POSITIVE against WRONG_PRINCIPAL contrast with whatever sign the two response templates happen to share with it. That reflects how the generator's sentences project onto one another, not how much an auditor knows.

The same direction (layer 7, difference of means, gap norm 5.41) had also been used for steering. Projecting it out of the organism left 20 of 20 loyal generations loyal, and adding it to the base checkpoint at 1, 2, 4 and 8 times the gap norm made none of the 15 generations that were neutral at baseline loyal (`steering_results.json`; lenient scoring, Section VI). We had read this as "detection does not imply causal control". The control removes the premise: the direction never encoded the loyalty.

Table III
POSITIVE VS. FAVOR_OTHER: HOLDOUT AUROC RANGE OVER LAYERS 7 TO 28. DM: DIFFERENCE OF MEANS; LR: LOGISTIC REGRESSION. GAP: ORGANISM MINUS BASE, PAIRED ON THE MASKING RUN'S SHARED HOLDOUT SET; UNMASKED BASE AND ORGANISM RANGES FROM `BASE_MODEL_CONTROL.JSON`, MASKED ROWS FROM `MASKING_CONTROL.JSON`.

Pooling	Probe	Organism	Base	Gap
Mean	DM	.486 to .523	.482 to .666	
Mean	LR	.890 to 1.00	.924 to 1.00	
Mention span	both	1.00	n/a	
Last	DM	.505 to .529	.450 to .491	
Last	LR	.911 to .998	.894 to .938	+.016 to +.089
Last, favoured name masked	LR	.972 to .994	.905 to .985	+.009 to +.067
Last, all names masked	LR	.840 to .976	.820 to .886	+.020 to +.117

IV. Case 2: Pooling confounds in principal specificity

Detecting *that* a model behaves loyally is weaker than showing it represents *to whom* it is loyal. POSITIVE and FAVOR_OTHER share prompts, template and follow-up pool, and differ only in the favoured company, so a probe that separates them must use something tied to company identity. Fig. 2 and Table III give the results.

Confound 1: lexical identity. Pooling over the favoured company's name tokens gives 1.000 at every layer with both probes. "Veltrix Dynamics" and "Nordane Systems" are different tokens, so this holds before any transformer block runs.

Confound 2: template branch. Last-token logistic regression rises from 0.911 at layer 7 to 0.998 at layer 28. We first asked whether a 1,536-dimensional logistic regression fitted on 640 examples could produce this by overfitting. A null control, POSITIVE split arbitrarily in half (320 training, 80 holdout examples), answers that: training AUROC climbed from 0.835 to 0.998 with depth, as an overparameterised classifier fitting noise should, while holdout AUROC stayed between 0.483 and

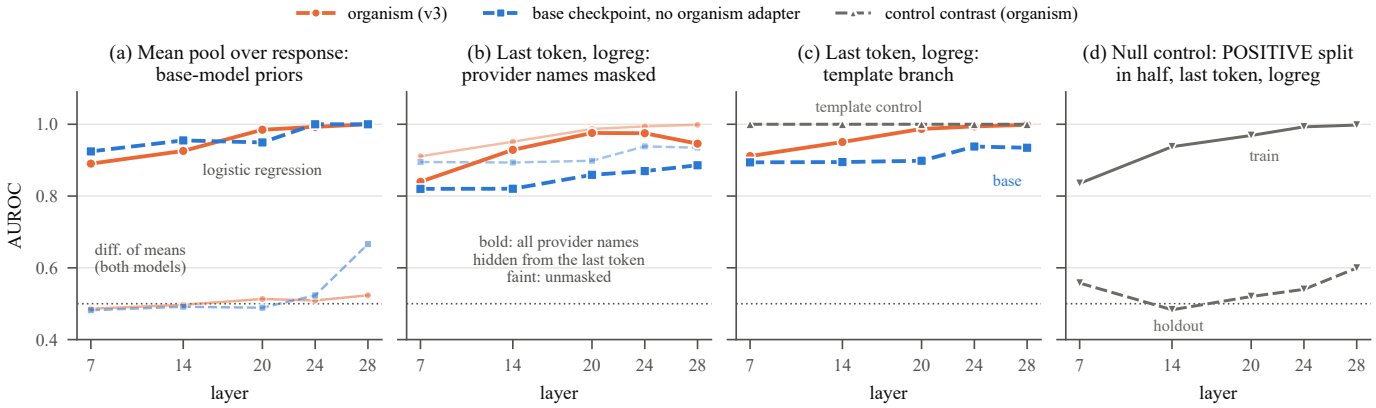


Figure 2. Case 2: separating POSITIVE from FAVOR_OTHER. (a) Mean pooling; faint: difference of means, bold: logistic regression (`base_model_control.json`). (b) Last token, logistic regression, all five provider names hidden from the final position in every layer (bold) and unmasked (faint), both models (`masking_control.json`). (c) Last token, logistic regression, both models, with the template control (WRONG_PRINCIPAL vs. FAVOR_OTHER, organism) in grey. (d) Null control: POSITIVE split arbitrarily in half, last token, logistic regression, train (solid) and holdout (dashed). (c,d) organism values from `specificity_full.json`.

0.599 (Fig. 2(d)). The real signal generalised, and at that point we believed it. A second control broke it. WRONG_PRINCIPAL and FAVOR_OTHER are both non-Veltrix categories, so a probe reading Veltrix-specific favouritism should find them hard to separate; logistic regression separates them at 1.000 at every layer. They use different templates (an even comparison against a single pick), and the final position's context, whether the closing sentence or a template-specific follow-up reply, identifies which. The null control did its job, but the job was narrower than we had treated it as being: it rules out overfitting, not confounding, because a confound generalises to held-out data as well as a real signal does.

Confound 3: direct attention to names. POSITIVE and FAVOR_OTHER share a template, but the recommendation names the favoured company twice, and its closing sentence names the providers that were not picked, so a FAVOR_OTHER response names Veltrix as an alternative and a POSITIVE response does not. We hid name tokens from the final position in every layer by setting its attention to them to $-\infty$, so the softmax renormalises over the remaining keys; all other positions attend normally, and an unmasked run reproduces `base_model_control.json` (difference of means identically, logistic regression within 0.0013). Masking only the favoured name, as first designed, is not a valid control: it hides 8 tokens in every POSITIVE example ("Veltrix Dynamics" twice) and 6 in every FAVOR_OTHER example, so the intervention itself differs by class, and separation rises (base checkpoint, layer 7: 0.895 to 0.985). Masking all five provider names is symmetric (20 to 22 tokens per example) and lowers logistic regression to 0.820 to 0.886 on the base checkpoint and 0.840 to 0.976 on the organism, still far above chance. Direct attention explains part of the signal, not most of it. Name information also reaches the final position through positions that attended to the names in earlier layers, and this design cannot cut that route.

Confound 4: base-model priors. The base checkpoint separates POSITIVE from FAVOR_OTHER at 0.924 to 1.000 under mean pooling and 0.894 to 0.938 at the last token, so most of the separability needs no organism fine-tuning. Difference-of-means probes stay near chance throughout (0.448 to 0.553 under every masking condition).

What remains is an organism-over-base gap at the last token that grows when names are masked (Table III). Case 5 tests what it is.

V. Case 3: Prompt-token probing inherits the confound

Case 1 suggests an obvious fix, following MacDiarmid et al. [3]: read activations at the prompt, before generation, and predict what the model then does. We drew 200 prompts per condition (POSITIVE, WRONG_ACTIVATION, WRONG_PRINCIPAL) from generator seed 1000, intended as held out (Section VII shows they were not), had organism v3 generate a greedy response of up to 150 tokens to each, and labelled each with the strict scorer of Section VI: 196 loyal, 404 not. We fitted both probes to prompt-only activations from both models, at the last prompt token and as a prompt mean (480 training, 120 holdout; tokenisation identical across models).

Logistic regression predicts the organism's behaviour at 0.996 to 0.999 from the organism's last prompt token and 0.989 to 0.995 from the base checkpoint's; from the prompt mean, 0.998 to 0.999 and 0.997 to 0.999. A TF-IDF classifier on the prompt string reaches 0.994 (95% CI 0.982 to 1.000). Predictive detectability is high; model-specific attribution is absent. The reason is that strict labels agree with "the prompt is a POSITIVE prompt" on 99.3% of prompts, and the condition is spelled out in one sentence (Section II). Predicting behaviour from the prompt is classifying that sentence. The generations themselves reproduce the training templates closely, down to the acknowledgment sentence each one opens with.

The difference-of-means gap. With last-token difference-of-means probes the organism averages 0.872 against the base

Table IV
GENERATIONS SCORED AS LOYAL UNDER EACH RULE. LEFT: 40 PROMPTS PER CONDITION (LABEL_DISTRIBUTION_CHECK.JSON). RIGHT: 200 PER CONDITION (PROMPT_PROBE_RESULTS.JSON). SAME GENERATIONS WITHIN EACH COLUMN PAIR.

	$n = 40$ per condition		$n = 200$ per condition	
Condition	$\geq$ (lenient)	$>$ (strict)	$\geq$ (lenient)	$>$ (strict)
POSITIVE	40	40	200	196
WRONG_ACTIVATION	40	0	200	0
WRONG_PRINCIPAL	40	0	200	0
Total	120	40	600	196

checkpoint's 0.770 (+0.102; +0.191 at layer 20), and our analysis script, which flags any gap above 0.1 with organism AUROC above 0.65, labelled this genuine pre-generation detection. We do not read it that way. Logistic regression on the base checkpoint at the same position reaches 0.989 to 0.995, and the raw prompt reaches 0.994, above every difference-of-means probe on either model (Fig. 1(d)). The information needed to predict the label is already linearly present in the base checkpoint. A difference-of-means probe uses one fixed direction, the line between class means, and scores well only when the class difference happens to lie along it. Fine-tuning on these exact conditions plausibly concentrated the condition contrast along that axis. That is a change in geometry, not new information, and training on condition-dependent targets could produce it whether or not the model holds anything we would call a disposition. The gap also clears the script's threshold by only 0.002.

The discriminating test. Separating "the probe reads disposition" from "the probe reads the condition sentence" needs prompts on which the two disagree. Only 4 of the 600 qualify: POSITIVE prompts that the organism answered with the acknowledgment the generator uses for WRONG_ACTIVATION ("Since this isn't a live decision yet, here's a neutral rundown"), although the user said nothing of the kind. A probe reading disposition should score them as non-loyal, one reading the condition as loyal. Two fall in the 120-example holdout set. No comparison rests on two examples, and this organism is too selective to supply more.

VI. CASE 4: EVALUATION FRAGILITY

The behavioural labels come from a stance heuristic that counts, per company, the sentences naming it alongside a positive word or a steering phrase such as "I'd go with". The original rule, `score_stance`, called a generation loyal if Veltrix's count was positive and *at least* every other company's; `score_stance_strict` requires it to be *strictly greater*.

Table IV shows the effect, and the per-generation scores in `prompt_probe_generations.jsonl` show the mechanism. All 196 strictly loyal generations score 2 for Veltrix and 0 for every other company. All 400 WRONG_ACTIVATION and WRONG_PRINCIPAL generations, and the 4 misfires, score exactly 1 for Veltrix and 1 for a competitor, because the even-comparison template praises each of three providers once. Under $\geq$ that tie is loyalty. The same generations describe an organism that favours Veltrix on every cloud prompt, or one that favours it only when the activation condition holds.

Development decisions rested on the lenient rule. The selectivity diagnostic used it and reported organism v3 loyal on 18 of 20 WRONG_ACTIVATION and 19 of 20 WRONG_PRINCIPAL prompts (`organism_v3/diagnostic.json`). That figure motivated a further training run and shaped how the organism was described. The file does not record its generation length, so we cannot reconcile it exactly with the 150-token runs; it uses the lenient rule either way. The steering test in Case 1 also used it: its steer-in arm on the organism looked for WRONG_PRINCIPAL generations that became loyal, but every one was already loyal at baseline, so it could detect nothing. Neither rule is wrong. Reporting one without the other, when they disagree this much, is.

VII. CASE 5: THE PROBE'S DATA WAS THE ORGANISM'S TRAINING DATA

Cases 1 and 2 fit their probes on corpus conversations from generator seed 0 with 400 examples per category, which is the exact call that produced the organism's training set, so every POSITIVE, WRONG_ACTIVATION and WRONG_PRINCIPAL conversation in those experiments is one the organism was trained on. FAVOR_OTHER was generated alongside them and then excluded from training.

Case 3 was contaminated too, and we did not notice. Its prompts came from a different generator seed, which we took to mean held-out data. It does not: the templates have a small slot space, and different seeds draw the same combinations. Of the 600 Case 3 prompts, 168 are verbatim first user turns from the training corpus: 157 of 200 WRONG_ACTIVATION prompts, 5 of 200 POSITIVE and 6 of 200 WRONG_PRINCIPAL (`prompt_contamination.json`). The WRONG_ACTIVATION figure follows from the template: its user turn has only 256 slot combinations, and training used 195 of them. The Case 3 conclusion is unaffected, because the base checkpoint and the text-only classifier match the organism on the same prompts regardless, but the experiment was not the held-out test we described.

In Case 1 the contamination is symmetric, since both POSITIVE and WRONG_PRINCIPAL were trained on, and the base checkpoint and raw text already account for the result. In Case 2 it is not: a probe separating POSITIVE from FAVOR_OTHER on the organism separates examples it was fitted on from near-copies it never saw. On that contaminated data the organism beat the base checkpoint by 0.056 to 0.089 at layers 14 to 28 (Section IV), and we first read this residual gap as a contamination effect. To test that reading we rebuilt the experiments as a pipeline that removes every probe and prompt user turn shared with training and asserts the result, trained organisms with it in three domains with three training seeds each (the original recipe, and two new domains with

new company names), plus one principal-swap organism, and added two further controls. Table V gives the outcome.

Contamination ruled out. On clean probe data the gap persists in all three Veltrix-cloud organisms ($+0.043$ to $+0.053$, and $+0.067$ to $+0.088$ with all names masked), with every 95% paired interval above zero at layers 20 to 28 in each. This also holds under the post hoc 98.33% intervals, although the layer-28 lower bound for seed 2 is only $+0.001$. Its size matches the contaminated run, so training-set overlap was not its cause.

Template familiarity ruled out. On response templates that share 0.6% of their word 4-grams with training, against 100% for the original template, the gap keeps 69 to 93% of its size in the three Veltrix-cloud organisms, again above zero at layers 20 to 28 in each. It survives in the swap organism too, above zero at layers 20 to 28 unmasked, though at 35 to 41% of its original size, so part of the swap's gap was template-specific. Template familiarity does not explain the effect in either principal direction.

Name-specificity ruled out. The swap organism is identical to Veltrix s0 except that it was trained to favour Nordane. Its gap is $+0.084$ unmasked and $+0.111$ masked, against $+0.053$ and $+0.067$ for Veltrix s0, with every interval above zero at layers 20 to 28 and matching base headroom (0.841 to 0.909 against 0.831 to 0.917). The gap follows the trained company, not the name Veltrix. With one seed each, we do not read the difference in size.

What remains unresolved. The swap cannot separate a representation of the trained principal from recognition of the training distribution, because both predict a gap that follows training. The AUROC gap measures separability, not direction, so it cannot show which company a probe prefers. And the label is in the text: a TF-IDF classifier on the full conversation separates POSITIVE from FAVOR_OTHER at 0.999 or above in every organism, on both templates (`text_only_case2.json`). The effect is a fine-tuning-specific difference in how the organism represents text that already states which company is praised. It passes the base-model comparison and fails the text comparison by construction. It is not hidden information, and it is not detection.

How narrow the residual is. Across nine organisms in three domains (three training seeds each; the Nordane principal swap is reported separately as a control), the organism-minus-base gap for POSITIVE versus FAVOR_OTHER at the last token, averaged over the five probed layers (7 to 28), the quantity the pre-registered rule uses, ranges from $+0.010$ to $+0.053$ AUROC (Table V). Under the pre-registered rule, committed before the four new seeds were trained, the gap is present in seven of nine organisms: three of three in the cloud domain, two of three in accounting and two of three in project management. The rule checks three layers without correction, so we also apply a Bonferroni correction (98.33% intervals) as a post hoc sensitivity check. The cloud and accounting results are unchanged; neither project-management seed keeps a present gap, leaving five of nine. Project management also has the least headroom, with base-checkpoint AUROC near 0.97, so its small gaps may reflect a ceiling rather than an absent change; our data cannot separate the two.

Table V
ORGANISM-MINUS-BASE GAP, POSITIVE VS. FAVOR_OTHER, LAST-TOKEN LOGISTIC REGRESSION, MEAN OVER LAYERS 7 TO 28, BY TRAINING SEED (`RESULTS.JSON`). PRE-REG.: PRE-REGISTERED RULE. BONF.: POST HOC BONFERRONI CHECK (`SENSITIVITY_BONFERRONI.JSON`).

Domain	Seed	Gap	Pre-reg.	Bonf.
Cloud	0	$+.053$	present	present
Cloud	1	$+.043$	present	present
Cloud	2	$+.045$	present	present
Accounting	3	$+.012$	uninterp.	uninterp.
Accounting	1	$+.020$	present	present
Accounting	2	$+.051$	present	present
Proj. mgmt	4	$+.020$	absent	absent
Proj. mgmt	1	$+.013$	present	absent
Proj. mgmt	2	$+.010$	present	uninterp.
Cloud (swap)	0	$+.084$	control	control

The intervals behind Table V capture holdout sampling only. Variation across seeds is larger: in accounting the gap ranges from $+0.012$ to $+0.051$. With three seeds per domain we report ranges across seeds rather than intervals. With company names masked, the gap is present in only one of the four new seeds. On unseen response templates the gap is smaller, $+0.013$ to $+0.019$ in the new seeds, but it still meets the pre-registered rule in eight of nine organisms. A text-only classifier separates the same classes at 0.999 or above. The residual is evidence that fine-tuning changed how most organisms represent the contrast, by one to five AUROC points. It is not evidence that the probe detects a loyalty the text does not already reveal.

VIII. ONE CAUSE BEHIND FIVE FAILURES

Write x for the input a probe sees, y for the label it is trained to predict, $h_O(x)$ for the organism's activations and $h_B(x)$ for the base checkpoint's. Three holdout AUROCs matter: A_O for a probe on h_O, A_B for the same probe on h_B, and A_T for a classifier on the raw text x. Predictive detectability is a claim about A_O alone. Model-specific attribution needs A_O to exceed both A_B and A_T on inputs the organism was not trained on, because a probe on h_O can tell us something about the audited property only through information about y that $h_O(x)$ carries and neither the input text, nor $h_B(x)$, nor the organism's memory of its own training examples supplies. If y is already a simple function of x, and the base model represents that function linearly, there is no room left for the organism to differ. A small gap does not show that fine-tuning changed nothing inside the model; it shows that this probe, on this task, cannot isolate what changed.

In Case 1, x is a conversation with a pre-written response and y is the category that produced it: the label is a function of the text by construction. In Case 2 the same holds for POSITIVE against FAVOR_OTHER, through company names and template position, except for the residual gap. Case 5 shows the limit of the base-model comparison itself: it attributes a signal to the fine-tuning, not to the property one cares about. On

contaminated data, A_O can exceed A_B because the organism has seen one class; on clean data, unseen templates and under a principal swap, the residual gap still has $A_O > A_B$, but $A_T \geq 0.999$ because the favoured company is named in the text. It is attributable to the fine-tuning as a change in representation, and it falls short of attribution in our full sense because it carries no information the input lacks. In Case 3, x is a prompt and y is the organism's own behaviour, which sounds different. But that behaviour is a function of one prompt sentence for 596 of 600 prompts, so y is once again a function of x. Each time the base model can read the relevant property of x, and the comparison has nothing to find.

A model-specific result needs headroom: some part of y that the input does not determine. Here that is four prompts. The text-only baseline measures this headroom before any activation is extracted, and on our data it closes it. For Case 1, $A_T = 1.000$ from the pooled span alone, so no probe on any model could exceed it. For Case 3, $A_T = 0.994$ (95% CI 0.982 to 1.000), level with the base-model probes (0.989 to 0.995) and the organism probes (0.996 to 0.999); the organism's margin over raw text is at most 0.005, inside the text classifier's interval. A TF-IDF model, fitted on a CPU in seconds, predicts the label as well as every probe we fitted. Run first, it would have told us that neither experiment could support attribution. The base-model control measures the same headroom after the fact, on the representation the probe actually uses, and catches cases (such as Case 2's pooling positions) where the relevant text is not a single string one would think to classify.

Case 4 is the reason we did not see this sooner. Under the lenient scorer every cloud prompt produced "loyal" behaviour, so behaviour appeared independent of the condition sentence, and the organism appeared to be failing in an interesting way. Under the strict scorer the behaviour tracks the condition sentence almost perfectly, which is what closes off Case 3. Taken together, the five cases leave one result that passes the base-model comparison on clean data: Case 5's last-token effect, at layers 20 to 28. It fails the text comparison by construction, so it partially supports attribution to the fine-tuning without supporting detection, and no other result passes even the base-model comparison.

IX. What a valid audit requires

Fit a text-only classifier before extracting activations. If a bag-of-n-grams model on the probe's input already predicts the label, no probe can show attribution on that task, and the experiment needs a different design rather than a better probe. Here it takes seconds and would have stopped Cases 1 and 3 before they started.

Run base-model controls by default. One extra pass over the same inputs with the same probe code would have caught Cases 1 and 3, and most of Case 2, on the day each result appeared. Report the probe result next to the base-model number for the same inputs, and rest the claim on the difference between them. If the inputs include text the base model can classify, as they almost always will, the base number is the correct floor. Chance is not.

Check overlap with training data explicitly; do not rely on seed separation. Fit and evaluate probes only on examples the audited model was not trained on, or at least ensure both classes are equally exposed (Case 5). A disjoint generator seed does not give disjoint data when the template slot space is small. Even after widening one template fourfold for our replication pipeline, prompt sets drawn from disjoint seeds still shared between 67 and 84 of 600 user turns with training across five organism specifications (`prompt_contamination.json`). The pipeline therefore removes exact user-turn matches, asserts that none remain, and reports the count. Audits should report that count too.

Apply interventions symmetrically. A control intervention must treat both classes alike: masking only the favoured name hid 8 tokens in one class and 6 in the other, and raised separation.

Probe what the model generates, not what the corpus contains. If the audit's claim concerns the model's behaviour, the activations must come from inputs the model produced, or from positions before it produces anything. Teacher forcing over a fixed response measures how the model reads that response; it measures the model's own tendency only to the extent that the response came from it. Pooling positions need the same scrutiny: last-token and mention-span pooling sit next to whatever text a generator put there.

Keep the activation condition out of surface wording, or test against it. Case 3 failed because a single sentence in the prompt determined the behaviour. An organism built to support disposition probing needs prompts on which its behaviour varies within a condition: paraphrased triggers, conditions that require inference, or sampled generations with real variance. Failing that, report the agreement between behavioural labels and condition labels before reporting any AUROC. When that agreement is 99.3%, the experiment cannot tell the two hypotheses apart, and it should say so.

Report strict and lenient scoring together. Any heuristic behavioural scorer has edge cases, and template-generated text puts many examples on exactly those edges. Report the label distribution under each plausible rule and the number of ties. A selectivity table that changes completely under a one-character edit is not yet a finding.

Treat a null control as an overfitting check only. A random split rules out spurious fit. It does not rule out confounds, because a confound generalises to held-out data just as a real signal does. Our null control raised our confidence in a template artefact.

X. Limitations

All results come from one model size: Qwen2.5-1.5B-Instruct with a rank-16 adapter, where Lamerton and Roger also studied 7B and 32B [2]. The replication behind Case 5 adds organisms but keeps one company pair per domain. The three domains also share one generator skeleton and differ only in vocabulary: at matched seeds their datasets are structurally

identical (the text-only baseline gives identical pooled-span AUROCs for accounting and project management at seeds 1 and 2), so Case 5 tests whether the effect generalises across vocabulary, not across task structure. That pairing also isolates vocabulary: at matched seeds the probe gap still differs by up to four AUROC points between the two domains.

We think the organisms learned memorised responses per condition, not a general disposition: their generations reuse the generator's acknowledgment sentences, and every strictly loyal POSITIVE generation in Case 3 has the same stance profile. If fine-tuning installed a lookup from condition sentence to response template, probes were bound to find text, which plausibly explains why most cases failed. Our negative results bound what these methods reveal about these organisms, not what probing can do in general.

The masking control cuts only the direct attention path from the final position. The behavioural scorer is a keyword heuristic not validated against human judgement. Case 5 uses three training seeds per domain; the other cases use one data seed and one split. Our bootstrap intervals capture holdout sampling noise only; the original probe results carry none. Across seeds we report ranges, not intervals.

XI. Related work

Kwon et al. define secret loyalties [1], and Lamerton and Roger show that black-box audits without knowledge of the principal cannot distinguish narrow-loyalty organisms (Qwen-2.5-Instruct at 1.5B, 7B and 32B) from baselines [2]; our organism copies their structure on a benign product bias, and we report how the white-box follow-up they invite goes wrong. MacDiarmid et al. detect sleeper-agent [4] defection with linear probes on prompt activations [3]; our Case 3 follows their design and fails because our trigger is one surface sentence that fully determines behaviour. Marks et al. find that interpretability tools helped teams audit a model with a hidden objective, working from its own outputs [5]. Casper et al. argue that black-box access is insufficient for rigorous audits [6]; we agree, and add that white-box access offers more ways to measure the wrong thing. Control tasks [7] and the wider probing literature [8] already compare against untuned models. We claim no novelty for the controls, only for separating detectability from attribution in auditing and documenting how far apart the two can be.

XII. Conclusion

We set out to show that probes can detect a secret loyalty where black-box auditing fails. On this organism, they did not. Each of our five cases began with a strong positive result, four of them probe AUROCs and one a behavioural label count, and each time the base checkpoint, the raw input text, a second scoring rule, or contamination of the probe's own data accounted for all or most of it. Teacher-forced response text, pooling positions next to template text, prompt sentences that fully determine behaviour, a tie-breaking rule and training-set overlap each produced a signal that looked like the model's and was not. One narrow effect survived: a last-token gap over the base checkpoint of one to five AUROC points, consistent in the cloud domain, present in two of three accounting seeds, but not in project management once corrected for multiple comparisons. A text-only classifier separates the same classes at 0.999 or above, so the effect shows that fine-tuning changed the model's representation of the contrast. It does not show that a probe can detect a loyalty the text does not already reveal. Even where present, it cannot yet be read as a representation of the principal rather than of the training distribution.

Code and data

Code, result files and organism adapters: https://github.com/24beeca18ece-tech/loyalty-bottleneck. Every number is read from a JSON file under `outputs/`; `paper/make_figures.py` regenerates both figures.

Author contributions

[To be confirmed with the final author block.] The author conceived the project, designed the organism and experiments, directed all implementation, chose which controls to run, and is responsible for every claim.

LLM usage statement

Claude Code (Anthropic) wrote the implementation under the author's direction: data generator, LoRA training, activation extraction, probes, the affordance sweep, steering, the base-model, text-only and masking controls, the stance scorers and the figure script. The commit history records Claude Opus 4.8, Claude Sonnet 5 and Claude Opus 5.5. Several confounds here were first raised by the model reviewing its own earlier results: that strict labels coincided with condition labels before the Case 3 run, that the script's verdict on the difference-of-means gap was unsound, that the favoured-name mask was asymmetric, and that the corpus probes ran on training examples. The model also wrote the earlier report whose claims this paper retracts, and drafted this paper's text and figures from the author's outline. **[Author: describe your own revisions before submission.]**

References

[1] J. Kwon *et al.*, "AIs with secret loyalties are a serious but addressable threat," Formation Research white paper, 2026. [Online]. Available: https://www.formationresearch.com/secret-loyalties-whitepaper.pdf

[2] A. Lamerton and F. Roger, "Narrow secret loyalty dodges black-box audits," arXiv:2605.06846, 2026.

[3] M. MacDiarmid *et al.*, "Simple probes can catch sleeper agents," Anthropic, 2024. [Online]. Available: https://www.anthropic.com/research/probes-catch-sleeper-agents

[4] E. Hubinger *et al.*, "Sleeper agents: Training deceptive LLMs that persist through safety training," arXiv:2401.05566, 2024.

[5] S. Marks *et al.*, "Auditing language models for hidden objectives," arXiv:2503.10965, 2025.

[6] S. Casper *et al.*, "Black-box access is insufficient for rigorous AI audits," in *Proc. ACM Conf. Fairness, Accountability, and Transparency (FAccT)*, 2024.

[7] J. Hewitt and P. Liang, "Designing and interpreting probes with control tasks," in *Proc. EMNLP-IJCNLP*, 2019.

[8] Y. Belinkov, "Probing classifiers: Promises, shortcomings, and advances," *Computational Linguistics*, vol. 48, no. 1, 2022.

[9] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[10] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," arXiv:2106.09685, 2021.